THE TURNSTILE-ORACLE CAUSAL RING

A decoupled synchronization and storage architecture, proved in Lean 4

BY

DOUGLAS QUIGG

RESEARCH AND THEORY
CONCURRENT SYSTEMS

EMBEDDED_TYPES
2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

ABSTRACT

Concurrent queues couple synchronization to storage: a compare-and-swap loop or a per-slot sequence counter guards every slot, and the proof of the queue is the proof of that coupling. We separate the two. A *turnstile* is a manifold of lanes, each one atomic word on its own cache line. A reader enters with one fetch-or, leaves with one fetch-add, and a writer drains by reading each open lane once and waiting on the busy ones. A *ticket ring* is a power-of-two array of slots that a 64-bit stamp indexes with one mask; its push and pop have no branch and no atomic operation, because the turnstile is the only guard. Every slot write records the stamp that wrote it, so the ring is a log of causal events as well as a queue.

The C11 implementation is seven headers. The Lean 4 formalization proves the lane word at 32 and 64 bits, the open count, the queue word, the backoff, the drain and the sleeps of the turnstile, and the stamp, the place, the window and the pushes of the ring. GenMC checks the drain and the resize of the manifold under five memory models. We state each theorem as it is checked, we name the trusted axioms (the atomics of the target, the futex contract of the host, the depth bound that the grantor of tickets keeps), and we state what the artifact does not provide: a rewind operation, a parity bit, and a sandbox case study are design arguments over the proved interface, not machine-checked results.

CONTENTS

1

THE TURNSTILE-ORACLE CAUSAL RING

This paper describes and verifies a decoupled synchronization and storage architecture. The synchronization half is the *turnstile*, a manifold of lanes in `include/embed_turnstile.h`. The storage half is the *ticket ring*, a power-of-two array of slots in `include/embed_ticket_ring.h`. Both are C11 headers of the embedded_types library ISO C11 [**?**]. The formalization is the Lean 4 project under `theory/embedded_types` Lean 4 [**?**], and the model checks are the GenMC clients under `test/prove` GenMC [**?**]. Every claim below is one of three kinds, and the paper names the kind each time: a theorem that Lean checks, an execution space that GenMC exhausts under a named memory model, or an axiom that the artifact trusts from the target, the host or the caller.

1.1 1. INTRODUCTION

A bounded concurrent queue is usually one object. The Vyukov multi-producer multi-consumer queue Vyukov [**?**] stores a sequence counter beside each slot, and a producer claims a slot with a compare-and-swap loop over the enqueue position, then publishes the slot by a release store of the counter. The LMAX Disruptor Disruptor [**?**] couples a ring, a sequence barrier and a wait strategy into one framework. In both, the proof of the data structure is the proof of the coupling: a slot is correct because the counter next to it says so, and the counter is correct because every path that writes the slot also writes the counter.

We take the opposite position. The ring holds no counter. A push is one shift, one mask and two plain stores. A pop is one shift, one mask and two plain loads. Neither has a branch and neither has an atomic operation. The guard is outside the ring: the turnstile. A worker enters a turnstile before it touches the ring and leaves after. The entry is one atomic fetch-or on a lane word, the leave is one atomic fetch-add on the same word. A writer that wants to retire storage swaps a published pointer and *drains*: it reads each open lane once, and waits on each lane that a pass holds until that pass ends. The drain is the only wait in the architecture, and the only place where one thread observes the progress of another.

The decoupling buys three things.

- **A small ABI.** The ring exports a stamp, a place, a push and a pop, and two wrappers that put the push and the pop inside a pass. The turnstile exports an entry, a leave, a drain, a lane count and a backoff. There is no wait strategy to configure and no sequence barrier to compose.

- **A small proof.** The ring is proved as a function from a list of pushes to a ring state, with no concurrency in the model, because the turnstile keeps each push and each pop inside a pass. The turnstile is proved as a transition system over one lane word, one open count and one queue word. The two proofs meet at one obligation: a pass is exclusive with respect to the drain that waits for it.
- **A causal log.** Every slot records the 64-bit stamp of the push that wrote it. A pop returns that stamp beside the job. The stamp is injective in its two fields, so a consumer can tell the ticket that wrote a slot from the ticket it expected, which turns the ring into an append-only record of causal events over its window.

The rest of the paper is the system model (Section 2), the Lean 4 formalization with its theorems (Section 3), a composition argument for a user-space sandbox over the interface (Section 4, which is a design argument and not a verified result), related work (Section 5), and the trust boundary (Section 6).

1.2 2. SYSTEM MODEL AND THE COMPOSITE WORDS

2.1 Two composite words

The architecture packs its state into two 64-bit words, each one a composite of fields.

The **lane word** is the state of one turnstile lane. From bit 0 up, its fields are:

bits	field	meaning
0	busy	a pass, a contraction or a growth holds the lane
1	closed	a contraction closed the lane; it lies at or above the open count
2	sleeper	a thread sleeps on the word with the futex contract
3 up	generation	one count for each pass end, each close, each release and each takeover; rolls at the width

The C defines the fields as `EMBED_TURNSTILE_BUSY`, `EMBED_TURNSTILE_CLOSED`, `EMBED_TURNSTILE_SLEEPER` and `EMBED_TURNSTILE_GENERATION`, with the generation shift at 3. Each lane is one `embed_atomic_word`, the atomic word as wide as the machine word, aligned to 64 bytes so that no lane shares a cache line with another:

```
typedef struct
{
    EMBED_ALIGN(64) embed_atomic_word word;
} EmbedTurnstileLane;
EMBED_STATIC_ASSERT(sizeof(EmbedTurnstileLane) == 64u, "a lane takes one cache line");
```

The **stamp** is the capability that the ring indexes by. It is one `embed_u64`, the epoch in the high 32 bits and the ticket in the low 32 bits:

```
#define EMBED_TICKET_BITS 32u
#define EMBED_TICKET_MASK ((embed_u64)0xffffffffu)
#define EMBED_TICKET_EPOCH_SHIFT 32u
```

```
EMBED_INLINE embed_u64 embed_ticket_stamp(embed_u32 epoch, embed_u32 ticket)
{
    return ((embed_u64)epoch << EMBED_TICKET_EPOCH_SHIFT) | (embed_u64)ticket;
}
```

The stamp holds no turnstile flag and no parity bit. The prompt that framed this work asked for a single word that carries a timestamp, waiter and parity flags and a ticket index. The artifact splits that role: the lane word carries the waiter flag (sleeper) and the hold flag (busy), and the stamp carries the epoch and the ticket. Section 6 returns to parity.

2.2 The ring

A ring is a mask and a pointer to 2^s slots that the caller owns:

```
typedef struct
{
    embed_u64 stamp;
    void *job;
} EmbedTicketRingSlot;
EMBED_STATIC_ASSERT(sizeof(EmbedTicketRingSlot) == 16u, "a slot is two words");

typedef struct
{
    EMBED_ALIGN(64) embed_u64 mask;
    EmbedTicketRingSlot *slots;
} EmbedTicketRing;

EMBED_INLINE void embed_ticket_ring_begin(EmbedTicketRing *ring, EmbedTicketRingSlot *slots, unsigned int shift)
{
    ring->mask = ((embed_u64)1u << shift) - 1u;
    ring->slots = slots;
}
```

The place of a stamp is its ticket masked to s bits. A push writes the slot at that place, a pop reads it:

```
EMBED_INLINE embed_u64 embed_ticket_ring_place(const EmbedTicketRing *ring, embed_u64 stamp)
{
    return stamp & EMBED_TICKET_MASK & ring->mask;
}

EMBED_INLINE void embed_ticket_ring_push(EmbedTicketRing *ring, embed_u64 stamp, void *job)
{
    EmbedTicketRingSlot *const slot = &ring->slots[embed_ticket_ring_place(ring, stamp)];

    slot->stamp = stamp;
    slot->job = job;
}

EMBED_INLINE void *embed_ticket_ring_pop(const EmbedTicketRing *ring, embed_u64 stamp, embed_u64 *stamp_out)
{
    const EmbedTicketRingSlot *const slot = &ring->slots[embed_ticket_ring_place(ring, stamp)];
```

```
    *stamp_out = slot->stamp;
    return slot->job;
}
```

The slot access is non-atomic by design. The C11 memory model gives these loads and stores no ordering of their own; the ordering comes from the turnstile around them (Section 2.4). The shift is at most `EMBED_TICKET_RING_SHIFT_MOST`, which is 31, so the mask never reaches the epoch.

2.3 The turnstile

A set of turnstiles is `EmbedTurnstiles`: `EMBED_TURNSTILE_LANES` lanes (64 by default, and a build can set 1 to 64), an open count `open` that holds the number of open lanes less one, and a queue word `queue` that counts the workers that found every open lane busy. A set of zeros is at rest with one lane open.

An **entry** by a worker at address a reads the open count n, starts at lane $(a \gg 6) \bmod n$, and tries a fetch-or of the busy bit on each lane in turn. The fetch-or takes the lane where the old value has busy clear; where the old value has busy set, the fetch-or changed nothing and the entry spills to the next lane modulo n. Where every open lane was busy, the worker adds one to the queue word and tries to grow the manifold: a compare-and-swap on lane n from free or from closed to busy, then a compare-and-swap of `open` from n to $n + 1$. A growth that lands gives the worker lane n as its pass. A growth that fails backs off (Section 2.5) and tries again. On its way out of the entry a worker that was in the queue subtracts one from the queue word, and the entry ends with a sequentially consistent fence.

A **leave** on lane k is one fetch-add of `GENERATION - BUSY`, which clears busy and adds one to the generation in a single indivisible step. Where the old value had the sleeper bit, the leave clears that bit with a fetch-and and wakes every thread asleep on the word. Where k was the top open lane, the leave contracts the manifold while the queue word reads zero, more than one lane is open and no pass holds the top lane. A contraction claims the top lane with the same fetch-or an entry uses, swaps `open` down by one, and closes the lane with a fetch-add of `GENERATION + CLOSED`. The chain of contractions after one leave is the *cascade*.

A **drain** by a writer runs after the writer has swapped a published pointer. It issues a sequentially consistent fence, reads the open count, reads each open lane once into a snapshot, and then for each lane whose snapshot has busy set and closed clear it waits until busy reads clear or the generation differs from the snapshot. Every pass that began before the fence of the drain and still holds a lane at the fence is below the open count that the drain reads (Theorem 3.3), so the snapshot covers every such pass, and the generation makes each wait end at the end of the pass that the snapshot saw and not at a later pass on the same lane.

2.4 Memory ordering

The atomics header selects one of four arms. On the C11 arm, a load is `memory_order_acquire`, a store is `memory_order_release`, every read-modify-write is `memory_order_acq_rel`, and the fence `EMBED_ATOMIC_ORDERED()` is `atomic_thread_fence(memory_order_seq_cst)`. The GCC and clang

builtin arm uses the same orders through `__atomic_*`. The MSVC arm uses the interlocked intrinsics, which are full barriers on x86 and x64, and a full barrier for the fence. The CUDA arm uses the device atomics and `__threadfence()`.

The ordering argument of the architecture is two fences. The entry of a worker ends with a fence after its fetch-or claims a lane, before the pass reads any data of the published pointer. The drain of a writer begins with a fence after the writer swapped the pointer, before the drain reads the open count and the lanes. With both fences sequentially consistent, every execution orders either the claim of the worker before the fence of the drain, in which case the drain reads the lane as busy and waits for the leave of that pass, or the fence of the drain before the claim, in which case the worker reads the new pointer. The slot stores inside a pass and the slot loads inside another pass are ordered through the acquire-release read-modify-writes on the lane words and the drain in between: a consumer pops a slot either inside a pass that the producer's drain waited for, or inside a pass that began after the producer's drain, and in the second case the acquire of its entry synchronizes with the release of the producer's leave through the lane word.

This argument is checked and not proved: Section 3.6 names the five memory models under which GenMC exhausts the executions of the drain and the resize clients, and the C11 model is among them RC11 [?].

2.5 The sleep and the futex contract

A worker that cannot grow the manifold, and a drain that waits on a held lane, back off on the lane word. The backoff keeps two locals, the nanosecond at its start and an exponent k, and no shared word holds a time. Inside the first unit of 2^u nanoseconds, with u as `EMBED_TURNSTILE_UNIT_SHIFT`, a step is a processor pause and no call. From one unit on, a step sets the sleeper bit with a fetch-or and calls the wait of the host on the lane word for at most 2^{u+k} nanoseconds, then k rises by one to at most `EMBED_TURNSTILE_K_MOST`, which is 20.

The host supplies three functions, and the headers declare them without a definition:

```
int embed_word_wait(embed_atomic_word *word, embed_atomic_word_value seen, embed_u64 nanoseconds);
void embed_word_wake(embed_atomic_word *word);
embed_u64 embed_clock_ns(void);
```

The contract of the wait is the futex contract futex [?]: the compare of the word against `seen` and the sleep are one step with respect to every wake on the word, and a wake ends each sleep on the word. The test host implements it on the Linux futex, on `WaitOnAddress` on Windows, on `__ulock_wait` on macOS, and as a timed sleep elsewhere. The futex compares 32 bits; on a little-endian host those are the low half of the lane word, which holds the three state bits and the low bits of the generation, and every leave, close and takeover changes them (Theorem 3.1, `changes_seen`). A build that defines none of the three functions fails at the link, and the error names them.

1.3 3. FORMAL VERIFICATION IN LEAN 4

The Lean project is `theory/embedded_types`. It builds with Lean `v4.35.0`, and the proofs workflow fails where any theorem closes with `sorry`. Bit-level theorems are stated on `BitVec` at 32 and at 64 bits and closed with `bv_decide`, which checks the statement on every value of the width through a

SAT solver with a verified certificate checker. Transition-system theorems are closed by induction over an inductive `Reach` relation. This section lists the definitions and the theorems as the files state them.

3.1 The lane word (`Manifold/LaneWord.lean`)

The operations of the C are definitions on `BitVec w`:

```
def busy (w : Nat) : BitVec w := 1
def closed (w : Nat) : BitVec w := 2
def sleeper (w : Nat) : BitVec w := 4
def unit (w : Nat) : BitVec w := 8
def generation {w : Nat} (x : BitVec w) : BitVec w := x >>> 3
def entered {w : Nat} (x : BitVec w) : BitVec w := x ||| busy w
def takes {w : Nat} (x : BitVec w) : Bool := x &&& busy w == 0
def left {w : Nat} (x : BitVec w) : BitVec w := x + (unit w - busy w)
def shut {w : Nat} (x : BitVec w) : BitVec w := x + (unit w + closed w)
def takenOver {w : Nat} (x : BitVec w) : BitVec w := x + (unit w - closed w)
def flagged {w : Nat} (x : BitVec w) : BitVec w := x ||| sleeper w
def unflagged {w : Nat} (x : BitVec w) : BitVec w := x &&& ~~~(sleeper w)
def low {w : Nat} (x : BitVec w) : BitVec w := x &&& 0xFFFFFFFF
```

Each theorem below exists at 64 bits and at 32 bits; the paper names the 64-bit form.

- `entry_takes_64`: $\forall x,\ x \wedge 1 = 0 \Rightarrow \mathrm{entered}(x) = x + \mathrm{busy} \ \wedge \ \mathrm{entered}(x) \wedge \neg\mathrm{busy} = x$. An entry on a free word sets exactly the busy bit and leaves every other field, the generation among them, as it was.
- `entry_refused_64`: $\forall x,\ x \wedge 1 = 1 \Rightarrow \mathrm{entered}(x) = x \wedge \neg\mathrm{takes}(x)$. An entry on a busy word changes nothing.
- `leave_64`: a leave on a word with busy set clears busy, keeps the closed and sleeper bits, and adds one to the generation.
- `close_64`: a close on a word with busy set and closed clear sets closed, keeps busy and the sleeper bit, and adds one to the generation.
- `takeover_64`: a takeover on a word with both bits set leaves the two low bits at busy alone, keeps the sleeper bit, and adds one to the generation.
- `changes_seen_64`: $\forall x,\ (x \wedge 1 = 1 \Rightarrow \mathrm{low}(\mathrm{left}(x)) \neq \mathrm{low}(x)) \wedge (x \wedge 3 = 1 \Rightarrow \mathrm{low}(\mathrm{shut}(x)) \neq \mathrm{low}(x)) \wedge (x \wedge 3 = 3 \Rightarrow \mathrm{low}(\mathrm{takenOver}(x)) \neq \mathrm{low}(x))$. Every operation that ends a hold, applied to a word in the state it applies to, changes the 32 bits that the futex compares.
- `sleeper_not_over_64` and `end_over_64`: the sleeper bit alone does not make a snapshot read as over, and a leave does.
- `pass_64`: $\forall x,\ x \wedge 1 = 0 \Rightarrow \mathrm{left}(\mathrm{entered}(x)) = x + \mathrm{unit} \wedge \mathrm{left}(\mathrm{entered}(x)) \wedge 1 = 0$. One pass adds exactly one unit to the word and returns the lane free.

- `roll_64`: $\forall x,\ x + \text{unit} \cdot 2^{61} = x$. The generation rolls after 2^{w-3} counts.
- `classes_64` and `class_two_never_64`: the two low bits give the three lane states of `Open.lean` (free, held, closed), and no operation reaches the fourth class, closed with busy clear.

Theorem 1 below is the conjunction that the architecture needs from this file: **a lane word is exclusive**. Where a worker's fetch-or returns a value with busy clear, no other fetch-or returned busy clear between that worker's entry and its leave, because `entry_refused` holds for every intermediate value and `leave` is the only operation that clears busy on a held lane. This is a property of the atomic read-modify-write on one word, which the target guarantees (Axiom A1) and the Lean file states per operation.

3.2 The open count and the manifold (`Manifold/Open.lean`)

The manifold is a state with an open count `count`, a high-water mark `high`, and a lane function:

```
inductive Lane where
  | free | pass | held | sealing | closed
  | growing (fromClosed : Bool)

structure State where
  count : Nat
  high : Nat
  lane : Nat → Lane
```

`Step most` is an inductive relation with one constructor for each atomic operation of the C: the fetch-or of an entry, the fetch-add of a leave, the fetch-or of a contraction and its swap of `open` that lands or fails, the fetch-add that closes a sealing lane, the compare-and-swap of a growth from free or from closed, and the swap of `open` that lands or fails. Other steps can come between any two of them. `Reach most` is the reflexive-transitive closure from `rest`, the set of zeros. An invariant `Inv most s` is proved for every reachable state, and from it:

- `start_in_range`, `start_shift`: $\forall a\, n,\ 0 < n \Rightarrow (a \gg 6) \bmod n < n$, and the C expression `(address >> 6) % n` is that start.
- `spill_lane`, `spill_reaches`: the next lane of the C, o after the top and else the lane above, visits $(f + j) \bmod n$ at step j, so a spill from any start reaches each lane below n within n steps.
- `cascade_bounded`: $\forall c,\ 1 \le c \Rightarrow 1 \le \text{cascade}(c) \le c$. The cascade of contractions after a leave ends within $c - 1$ steps and never closes the last lane.
- `passes_below_open` (**Theorem 3.3**): for every reachable state s,

$$(\forall l,\ s.\text{lane}(l) = \text{pass} \Rightarrow l < s.\text{count}) \ \wedge\ (\forall l,\ s.\text{lane}(l) = \text{held} \Rightarrow l < s.\text{count}),$$

$$(\forall l,\ s.\text{lane}(l) = \text{closed} \Rightarrow s.\text{count} \le l) \ \wedge\ 1 \le s.\text{count} \le \text{most}.$$

Theorem 3.3 is what makes the snapshot of the drain complete: the drain reads `open` after its fence, and every lane that a pass can hold is below that value.

3.3 The queue word (`Manifold/Queue.lean`)

The queue word counts the workers between their addition and their subtraction. `queue_counts` proves $s.\text{word} = s.\text{waiting} \wedge 0 \leq s.\text{word}$ for every reachable state, and `empty_at_zero` proves that a word read at or below zero has no worker in the queue, which is the condition under which a leave contracts. `entry_balanced` proves that the entry of the C, with its local flag `queued`, adds at most once and subtracts only its own addition, so an entry that takes a lane leaves the queue word as it found it.

3.4 The drain and the sleeps (`Manifold/Drain.lean`, `Manifold/Wake.lean`)

The drain file models one lane with a holder, an ended set, and the busy, closed and generation fields. Its invariant is that a holder implies busy set and closed clear, and closed implies busy. The theorems:

- `held_until_end`: from a state where pass p holds the lane, each later state either has p in the lane with busy set, closed clear and the generation unchanged, or has p ended.
- `over_means_ended`: a drain that read p in the lane and later reads the lane over, where $\text{over}(\mathit{snapshot}, \mathit{now}) = \neg \mathit{now}.\text{busy} \vee \mathit{now}.\text{generation} \neq \mathit{snapshot}.\text{generation}$, saw p end.
- `drain_lane_complete`: with p the holder at the fence of the drain, a later read t, and a read u at which either t was free or closed or $\text{over}(t, u)$, the pass p has ended at u.

With Theorem 3.3, the loop of the drain over the open lanes ends with every pass of the fence ended. This is the correctness of the drain as a grace period in the sense of read-copy update RCU [**?**]: after the drain returns, no pass that began before the fence can still read the old pointer.

The wake file models the sleeper bit, a pending clear and an owed wake. Its theorem `no_lost_wake` states that in every reachable state a thread asleep on a value t has the word still at t with busy set, or a fetch-add owes it a clear and a wake, or the wake is out. A sleeper therefore sleeps only while the pass it waits for holds the lane, and the end of that pass wakes it. The theorem rests on the futex contract (Axiom A2): the compare and the sleep of the host are one step with respect to the wake.

3.5 The backoff (`Manifold/Backoff.lean`)

`shift_defined` proves that the shift `(embed_u64)1u << (EMBED_TURNSTILE_UNIT_SHIFT + exponent)` is below 64 bits for every exponent the C reaches, so the C is defined. `limit_fits` and `ceilings` bound every wait at 2^{30} nanoseconds on a CPU host and 2^{20} on a device. `no_call_in_unit` and `wait_from_unit` prove that a step inside the first unit makes no call and a step from one unit on waits with the limit of k. `timespec_fits`, `microseconds_fit` and `milliseconds_fit` prove that each host conversion of the limit fits its argument type. `update_is_next` proves that the branchless increment `exponent += (exponent < EMBED_TURNSTILE_K_MOST)` is the function `next K`.

3.6 The ticket ring (`TicketRing.lean`)

The stamp and the place are definitions on `BitVec` and on `Nat`:

```
def stamp (epoch ticket : BitVec 32) : BitVec 64 :=
  (epoch.zeroExtend 64 <<< 32) ||| ticket.zeroExtend 64
def ticketOf (s : BitVec 64) : BitVec 32 := (s &&& 0xffffffff#64).truncate 32
def epochOf (s : BitVec 64) : BitVec 32 := (s >>> 32).truncate 32
def place (shift ticket : Nat) : Nat := ticket % 2 ^ shift
```

Theorem 1 (non-aliasing). The theorems `stamp_ticket`, `stamp_epoch` and `stamp_injective` state

$$\forall e\, t,\ \mathrm{ticketOf}(\mathrm{stamp}(e,t)) = t \ \wedge\ \mathrm{epochOf}(\mathrm{stamp}(e,t)) = e,$$

$$\forall e_1\, t_1\, e_2\, t_2,\ \mathrm{stamp}(e_1,t_1) = \mathrm{stamp}(e_2,t_2) \Rightarrow e_1 = e_2 \wedge t_1 = t_2.$$

The theorems `place_lt`, `place_wrap`, `place_roll`, `place_mask` and `window_distinct` state

$$\forall s\, t,\ \mathrm{place}(s,t) < 2^s,$$

$$\forall s\, t,\ \mathrm{place}(s, t + 2^s) = \mathrm{place}(s,t),$$

$$\forall s\, t,\ s \le 32 \Rightarrow \mathrm{place}(s, t \bmod 2^{32}) = \mathrm{place}(s,t),$$

$$\forall s\, t,\ t \wedge (2^s - 1) = \mathrm{place}(s,t),$$

$$\forall s\, t_1\, t_2,\ t_1 < t_2 \wedge t_2 - t_1 < 2^s \Rightarrow \mathrm{place}(s,t_1) \neq \mathrm{place}(s,t_2).$$

The last of these is the non-aliasing theorem: two tickets fewer than 2^s apart stand at places of their own. The ring itself is a function from places to slots, and the pushes of a list are a fold:

```
def push (shift : Nat) (r : Ring) (ticket job : Nat) : Ring :=
  fun p => if p = place shift ticket then ⟨ticket, job⟩ else r p
def pop (shift : Nat) (r : Ring) (ticket : Nat) : Slot := r (place shift ticket)
def pushes (shift : Nat) (r : Ring) : List (Nat × Nat) → Ring
  | [] => r
  | (t, j) :: rest => pushes shift (push shift r t j) rest
```

`pop_push` proves $\mathrm{pop}(\mathrm{push}(r,t,j),t) = \langle t, j\rangle$, `pop_push_other` proves that a push at another place leaves a pop as it was, and `pop_pushes` proves

$$\forall l\, t\, j,\ (t,j) \in l \ \wedge\ \mathrm{Pairwise}_{\neq}(l) \Rightarrow \mathrm{pop}(\mathrm{pushes}(r,l),t) = \langle t, j\rangle,$$

where $\mathrm{Pairwise}_{\neq}(l)$ states that any two members of l have tickets at distinct places.

Theorem 2 (order independence of a window). `pop_pushes` quantifies over every list l whose tickets stand at distinct places, and places no condition on the order of l. Any two interleavings of the pushes of one window are two such lists with the same members, and the theorem gives the same pop for every ticket of the window under both. The ring state after a window is therefore a function of the set of pushes and not of their schedule. The theorem does not model two pushes to the same place, because the grantor keeps the depth of the ring below 2^s (Axiom A3), and it does not model a pop that interleaves with the push of its own ticket, because the turnstile keeps each push and each pop inside a pass and the drain of the producer orders the two (Section 2.4). The prompt that framed this work named this property Church-Rosser confluence of schedule projections; the artifact proves the statement above and no stronger one.

Theorem 4, first half (index containment). `place_lt` holds for every natural number t, with no hypothesis on t. A stamp whose ticket field a single-event upset flipped in any bit still indexes a slot below 2^s, because the mask of the C is the modulus (`place_mask`). The slot access is therefore in bounds for every 64-bit value of the stamp, flipped or not. The second half of the theorem, detection, is Section 6.

3.7 Model checks (`test/prove`)

Two GenMC clients run under the sequentially consistent, TSO, RC11, IMM and ARMv8 models. The drain client runs two workers through a set of one lane on one start lane while a writer swaps a published word and drains; the second worker finds the lane busy and grows the set. GenMC exhausts every execution the model allows and reports a failed assertion, a data race or a liveness violation. The assertions are that a worker that reads the old word reads it before the writer releases it, that no thread stays asleep with no wake to come, and that each lane is free or closed at the end. The resize client runs two workers through two passes each while a writer drains, so each growth races each contraction, and asserts that each pass holds a lane below the open count, that the drain ends, and that at the end no lane below the open count is closed or busy. A client that passes under one model and fails under another names the order that the code needs; both clients pass under all five.

3.8 Theorem 3, bounded reversibility

The prompt that framed this work asked for a proof of bounded rewind within a retention window W. The artifact does not define a rewind or an undo operation, in C or in Lean, and the paper claims none. What the artifact provides is the substrate: the ring retains the last 2^s pushes by place (`window_distinct`), each slot records the stamp that wrote it (`pop_push`), and the stamp is injective in its fields (`stamp_injective`). A consumer that holds a ticket t and a current ticket c with $c - t < 2^s$ can therefore pop the slot of t and confirm, from the stamp it reads back, that the slot is the push of t and not of a later ticket at the same place. A rewind over that window is a loop of such pops, and its correctness is `pop_pushes` applied to the window. The loop is not written and not verified.

1.4 4. CASE STUDY: A USER-SPACE SANDBOX OVER THE INTERFACE

This section is a design argument. Nothing in it is implemented in the artifact, and nothing in it is verified. The prompt that framed this work asked how page faults, inter-process communication and scheduling compose over a verifiable ring extension typeclass. The artifact has no such typeclass; `embed_ticket_ring.h` is a plain C interface and `TicketRing.lean` is a plain module. The argument below shows what that interface already fixes for such a sandbox and what a sandbox must add.

A user-space fault handler, such as one built on `userfaultfd`, receives fault events from the kernel and resolves each one by a copy or a mapping. Each event is a job. A grantor thread that owns the fault file descriptor grants one ticket per event in arrival order, inside a pass through the turnstile over the ring, and pushes the event with `embed_ticket_ring_pushed`. Each resolver thread pops the event of its ticket with `embed_ticket_ring_popped`, inside its own pass, and resolves it. The properties that the interface fixes:

- Each event has one slot while fewer than 2^s events are outstanding (`window_distinct`), and each resolver reads the event of its ticket (`pop_pushes`).
- The grantor retires a batch of resolved events by swapping the pointer to the slot storage and draining; after the drain, no resolver reads the old storage (`drain_lane_complete` with `passes_below_open`).
- A resolver that must wait sleeps on a lane word under the futex contract, and the end of the pass it waits for wakes it (`no_lost_wake`).

What a sandbox must add, and what this paper does not verify: the grantor must keep the number of outstanding events below 2^s, which is Axiom A3; the kernel must deliver each fault once, which is a property of `userfaultfd` and not of the ring; and the scheduling of resolvers to tickets is a policy outside the interface. Inter-process communication through the ring needs the slot storage in shared memory and the lane words in shared memory, and the futex contract across processes, which the Linux futex provides for a shared mapping and which the Lean model does not distinguish from the single-process case. A proof of the sandbox would be a refinement of its event loop onto the list-of-pushes model of Section 3.6, with the kernel events as the list, and would inherit `pop_pushes` for the storage and `drain_lane_complete` for the retirement. That refinement is future work.

1.5 5. RELATED WORK

Vyukov's bounded MPMC queue Vyukov [?] stores a sequence number in each cell. A producer loads the enqueue position, loads the sequence of the cell, and claims the cell with a compare-and-swap on the position when the sequence matches; it then stores the payload and publishes by a release store of the sequence. The queue needs no external guard and admits concurrent producers and consumers on distinct cells. The cost is one compare-and-swap loop per operation and one atomic per cell, and the proof of the queue is the proof of the cell protocol. The ticket ring has no per-slot atomic and no loop; it moves the loop into the entry of the turnstile, where a spill over

the lanes replaces a retry over the position, and it moves the per-slot sequence into the stamp that the push records. The ring does not admit two producers to one slot at all; it requires the grantor to keep the window.

The LMAX Disruptor Disruptor [?] is a ring with a monotonic sequence, a sequence barrier and a wait strategy, and its slots are pre-allocated objects. Its claim is a compare-and-swap or a single-writer increment on the cursor, and its consumers track the cursor through the barrier. The turnstile has no cursor; the only shared counters are the open count and the queue word of the manifold, and a reader never reads a producer's position. The Disruptor's wait strategies are a configuration surface; the turnstile has one backoff, bounded by the theorems of Section 3.5, and one host contract.

The Linux printk ring buffer printk [?] is a lockless ring with a descriptor ring and a data ring, with state transitions on descriptors by compare-and-swap, and it tolerates a writer that dies mid-record. The ticket ring does not tolerate a dead holder: a pass that never leaves holds its lane and the drain waits without end (Section 6). The printk design pays for that tolerance in the descriptor protocol; the ticket ring pays for its simplicity in the trust it places in the caller.

io_uring io_uring [?] is a pair of single-producer single-consumer rings between a process and the kernel, with head and tail indices and a mask over a power of two. The mask over a power of two is the same device as `place_mask`. io_uring has one producer and one consumer per ring by construction; the ticket ring admits many of each, and the turnstile is what pays for that.

seL4 seL4 [?] is a verified kernel with a proof of functional correctness from the specification to the binary. The artifact here is not a kernel and makes no such claim. Its proofs are of a data structure and a synchronization primitive, and Section 4 argues that a kernel-adjacent component can be refined onto them. **MCS locks** MCS [?] give each waiter its own cache line and hand the lock down a queue; the turnstile gives each lane its own cache line, but a lane is not a lock: many passes proceed at once on distinct lanes and the only thread that waits for a pass is a drain. In that respect the turnstile is closer to read-copy update RCU [?], with the drain as the grace period and the lane word as the per-reader state that the grace period waits on.

1.6 6. LIMITATIONS AND CONCLUSION

The trust boundary of the architecture is three axioms, one per source, and the paper has named each where it is used.

- **Axiom A1, the atomics of the target.** Each `embed_atomic_*` operation is one indivisible step with the memory order of its arm, and the sequentially consistent fence orders as C11 states. A target with none of the four arms fails the build at `#error`. The Lean model takes each operation as one step and does not model the compiler or the hardware.

- **Axiom A2, the wait of the host.** `embed_word_wait` compares and sleeps as one step with respect to `embed_word_wake`, and a wake ends every sleep on the word. The Lean theorem `no_lost_wake` and the GenMC liveness check both assume this contract. A host whose wait is a timed sleep, the fallback arm, keeps safety and loses the bound on the sleep.

- **Axiom A3, the depth bound.** The grantor of tickets keeps the number of tickets between the oldest unpopped and the newest granted below 2^s, and grants each ticket once. `window_`

`distinct` and `pop_pushes` are conditional on it. The ring detects a violation only through the stamp (below), and the turnstile does not detect it at all.

Beyond the axioms, the artifact has the following limits.

- **Single-bit upsets.** `place_lt` keeps every stamp, flipped or not, inside the slot array; that is Theorem 4 in full for the index. Detection of a flipped stamp rests on `stamp_injective`: a pop returns the stamp of the push that wrote the slot, and a consumer that compares it with the stamp it expected detects any flip in the stamp of the slot, or any flip in its own ticket that moved it to a slot of another ticket. The stamp carries no parity bit, so a flip in the job pointer of a slot is not detected by the ring, and a flip in the ticket of a consumer that lands on a slot the consumer also owns is not detected either. Multi-bit flips that preserve a valid stamp are outside every guarantee here.
- **A pass that never leaves.** The drain waits for every pass that holds a lane at its fence, without a timeout, and a lane has no owner that the drain could inspect. A reader that enters and never leaves blocks every later drain. The architecture trusts its readers.
- **Fixed retention.** The ring holds 2^s slots over storage that the caller owns, with $s \leq 31$. There is no growth of the ring, and the manifold grows and contracts only its lanes, to at most `EMBED_TURNSTILE_LANES`.
- **The generation roll.** The generation rolls after 2^{w-3} counts (`roll_64`). A drain that reads one lane twice with exactly that many pass ends between the two reads waits for one more pass to end. The header states this as a warning.
- **The device arm.** Under nvcc one definition compiles into the host pass and the device pass. The device arm of the atomics is checked by the device suite where a CUDA part is present, and the Lean and GenMC results do not cover the CUDA memory model.
- **What is not provided.** No rewind or undo operation, no parity bit, and no sandbox implementation exist in the artifact. Sections 3.8 and 4 state what the proved interface fixes for each and what remains to be built and proved.

In sum: the turnstile and the ticket ring separate the guard from the storage so that each can be proved on its own. The lane word, the open count, the queue word, the backoff, the drain and the sleeps are theorems of Lean 4 over a transition system whose steps are the atomic operations of the C. The stamp, the place, the window and the pushes of the ring are theorems over a function from places to slots, and the window theorem is what makes the ring a causal log. The two proofs meet at one obligation, that a pass is exclusive with respect to the drain, and GenMC checks that meeting point under five memory models. Everything beyond that is an axiom, named, or future work, named.

BIBLIOGRAPHY

[1] ISO/IEC. *ISO/IEC 9899:2011, Programming languages, C*. International Organization for Standardization, Geneva, 2011.

[2] L. de Moura, S. Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE 28, LNCS 12699, 625–635, 2021, doi:10.1007/978-3-030-79876-5_37.

[3] M. Kokologiannakis, V. Vafeiadis. *GenMC: A Model Checker for Weak Memory Models*. CAV 2021, LNCS 12759, 427–440, 2021, doi:10.1007/978-3-030-81685-8_20.

[4] D. Vyukov. *Bounded MPMC queue*. 1024cores.net, 2011. https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue.

[5] M. Thompson, D. Farley, M. Barker, P. Gee, A. Stewart. *Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads*. LMAX technical paper, 2011.

[6] J. Ogness. *printk_ringbuffer: a lockless ringbuffer for printk*. Linux kernel, kernel/printk/printk_ringbuffer.c and its header comment, 2020.

[7] J. Axboe. *Efficient IO with io_uring*. 2019. https://kernel.dk/io_uring.pdf.

[8] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, S. Winwood. *seL4: Formal Verification of an OS Kernel*. SOSP 2009, 207–220, doi:10.1145/1629575.1629596.

[9] U. Drepper. *Futexes Are Tricky*. Red Hat, 2011. Also `futex(2)`, Linux Programmer's Manual.

[10] P. E. McKenney, J. D. Slingwine. *Read-Copy Update: Using Execution History to Solve Concurrency Problems*. Parallel and Distributed Computing and Systems, 509–518, 1998.

[11] J. M. Mellor-Crummey, M. L. Scott. *Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors*. ACM TOCS 9(1), 21–65, 1991, doi:10.1145/103727.103729.

[12] O. Lahav, V. Vafeiadis, J. Kang, C.-K. Hur, D. Dreyer. *Repairing Sequential Consistency in C/C++11*. PLDI 2017, 618–632, doi:10.1145/3062341.3062352.